

A Policy-Driven Approach for Securing Microservices Workflow in Kubernetes Cluster

Bilal Ashraf, Satish Kumar, Nawar Jawad
School of Built Environment, Engineering and Computing
Leeds Beckett University, Leeds UK

b.ashraf2495@student.leedsbeckett.ac.uk, s.kumar, n.jawad@leedsbeckett.ac.uk

Abstract—Kubernetes based microservices deployments are increasingly adopted in modern cloud computing environments. However, securing microservices in Kubernetes cluster is critical because they operate in distributed manner across different nodes and each of which gives a potential entry point for attackers due to weak authentication mechanism or misconfiguration of security policies. To address these issues, we propose two layered policy-driven security mechanism for securing microservices workflow in Kubernetes cluster. The first layer employs mutual TLS (mTLS) for secure service-to-service authentication using the Istio service mesh. The second layer introduces workflow-based authorization policy enforced through Open Policy Agent (OPA) Gatekeeper. Experimental results demonstrate that unauthorized access is effectively blocked at the entry point, confirming that the proposed approach establishes a robust, multi-layered security environment for Kubernetes-based microservices.

I. INTRODUCTION

The modern microservices application runs in distributed fashion across different nodes in Kubernetes Cluster¹, typically built on top of virtual machines (nodes) in cloud data centre. The Kubernetes security by default is liberal unless properly configured, where each microservice in the cluster can communicate with other microservices without restrictions which expose vulnerabilities and potential entry point for the attackers to compromise one microservice (Pod) and gain access to the entire application running in cluster [1]. Therefore, securing microservices in a Kubernetes cluster is essential, specifically to implement security measures at individual microservice level. Kubernetes offers built-in security measures such as Role-Based Access Control (RBAC), network policies and Pod Security Policies. However, RBAC only manage end-user authentication, specifically who can access the resources (microservices) in the cluster, yet does not control how microservices securely communicate with each other after deployment [2]. This insecure microservice-to-microservice communication increases the chances for attackers to succeed. Therefore, it is important to ensure that a microservice authenticates any microservices it interacts with during operation, trusting them only after they present a verifiable identity document [3].

To address above issues, we employ Istio Service Mesh that provides built-in secure service-to-service communication. The Service mesh enables the PeerAuthentication between services by configuring sidecar proxies which are injected in every Pod

¹In a Kubernetes Cluster, each microservice is packaged as container which is then deployed and run into a Pod.

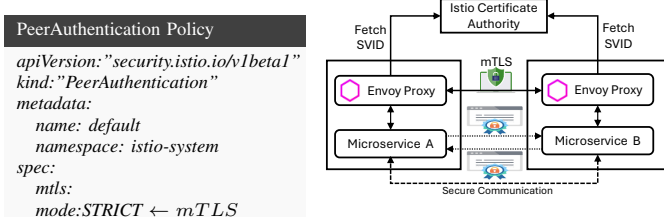


Figure A: Secure microservices communication

(microservices) running in Kubernetes Cluster. As we can see from Figure A, the traffic between microservices that have the sidecar proxies injected is mutually authenticated by sharing SVID certificate issued by Istio Certificate Authority.

To apply this policy in Kubernetes Cluster, we configure the service mesh to allow only mutually authenticated traffic encrypted with mutual TLS as shown in the left side PeerAuthentication Policy configuration file. However, manual configuration of policies can create chances for attackers to succeed if they are not properly configured like misconfiguration of mTLS mode in *PERMISSIVE* that allows unencrypted traffic which can expose service data or by compromising a pod in cluster, which exposes the service account token to give legitimate access to other service. Consequently, mTLS alone is insufficient to ensure a strict zero-trust policy for microservices workflow in Kubernetes cluster. To address this issue, we propose a two-layered security mechanism for securing microservice workflow in Kubernetes Cluster.

II. SYSTEM DESIGN AND IMPLEMENTATION

We employ Open Policy Agent (OPA) Gatekeeper [1] for creating a secure communication policy for each microservice in the workflow, which is further enhanced by integrating with Istio Service Mesh for building robust two-layered security mechanism for microservices workflow in Kubernetes cluster. As shown in Figure 1, the integration of OPA with Istio Service Mesh can be used as an authentication service to enforce security policies over microservice requests received by Istio sidecar proxy [4]. Aforementioned, Istio secure the runtime layer by managing microservice-to-microservice authentication (mTLS) and traffic encryption, while OPA Gatekeeper operates at admission control level for enforcing cluster wide policies for resources (microservice) validation. Therefore, we implement a cluster-wide workflow constraint policy named `workflowLabel` for creating second layer secure authentication for microservices workflow. This policy allows only those microservices labelled with `workflowLabel` have

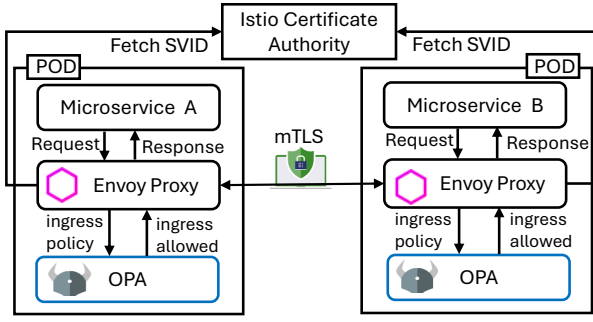


Fig. 1. Two-layered Security by integrating OPA with Istio Service Mesh

the legitimate access to communicate with each other in the cluster. We used OPA Constraints Template for defining our policy constraints named `workflowLabel` using Custom Resource Definition (CRD) as shown in ConstraintTemplate Policy box. And then, we create Constraint Policy that applies the previously created ConstraintTemplate with specific parameters to enforce the rules during microservice deployments as shown in Constraint Policy box.

OPA ConstraintTemplate Policy

```
apiVersion: templates.gatekeeper.sh/v1
kind: ConstraintTemplate
metadata:
  name: workflowLabel
spec:
  crd:
    spec:
      names:
        kind: workflowLabel
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego:
        package: workflowLabel
        violation["msg": msg]
        not input.review.object.metadata.labels["workflow"]
        msg := "Workflow label is required"
```

Constraint Policy

```
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: workflowLabel
metadata:
  name: require-workflow-label
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
  parameters:
    labels: ["workflow"]
```

III. RESULTS DISCUSSIONS

Experiment Setup: We design an experimental platform by deploying a Kubernetes Cluster on AWS Cloud, configured with one master node and two worker nodes. Further, we deploy Istio Service Mesh and OPA Gatekeeper in Kubernetes cluster for implementing secure microservice workflow policies for Google Online Boutique e-commerce application which contains 11 microservices [5].

Results: We used Google Online Boutique as an representative example for creating secure microservices workflow. We first configure the `PeerAuthentication` policy for creating a mutual mTLS authentication for entire online boutique microservices and then implement OPA based workflow

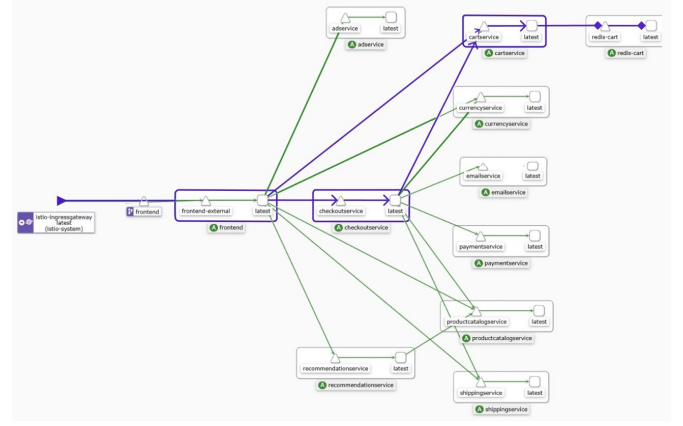


Fig. 2. Policies based secure microservices workflow

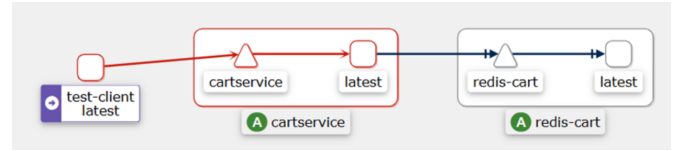


Fig. 3. Blocking unauthorized access of secure microservices workflow

constraint policy that provides policy based label, named `workflow`. We labeled only three microservices (`frontend`, `cartservice` and `checkout`) with `workflow` label from Google Boutique microservices and integrate them with Istio authorization policy. From Fig. 2, we can clearly see that after applying these policies, only `frontend` microservice communicates with the `cartservice` and `checkout` (purple line). Furthermore, we deploy `test-client` microservice without our policy, with an intension to access the `cartservice` in the cluster. From Fig. 3, we can see that our secure workflow policies block the `test-client` microservice access and ensure secure microservices execution in Kubernetes cluster.

IV. CONCLUSIONS

Kubernetes-based microservices face critical security risks due to distributed deployment and weak authentication. To address this, we propose a two-layered secure authentication by implementing Istio mTLS for secure service communication and OPA Gatekeeper for workflow-based authorization. Experimental results confirm our approach effectively blocks unauthorized access, establishing a secure workflow communication for microservices in Kubernetes cluster.

REFERENCES

- [1] R. Vadisetty, A. Polamarasetti *et al.*, "Ai-powered policy management: Implementing open policy agent (opa) with intelligent agents in kubernetes," *Cuestiones de Fisioterapia*, vol. 54, no. 5, pp. 19–27, 2025.
- [2] R. Sharma and A. Singh, *Getting Started with Istio Service Mesh: Manage Microservices in Kubernetes*. Apress, 2019.
- [3] J. Ray, *Policy as Code: Improving Cloud Native Security*. " O'Reilly Media, Inc.", 2024.
- [4] S. Pallewatta and M. A. Babar, "Towards secure management of edge-cloud iot microservices using policy as code," in *European Conference on Software Architecture*. Springer, 2024, pp. 270–287.
- [5] G. Inc., "Google online boutique microservice." <https://github.com/GoogleCloudPlatform/microservices-demo>, 2025, accessed: 2025-05-22.